Pearl Reference Manual and User's Guide

Bruno Bentzen[1]

[1]Zhejiang University,
Hangzhou, 310058, P. R. China
bbentzen@zju.edu.cn

Abstract

Pearl is an experimental proof assistant for the formalization of mathematics in line with the computational tradition established by Constable, Bickford, Allen, Harper, Rahli, Sterling, Angiuli and many others. Pearl implements the dialect of computational type theory systematically developed by Bentzen [9]. Although inspired by NuPRL and its descendents, Pearl is not oriented around a proof refinement logic for the demonstration of judgments. Instead, Pearl focuses on the manipulation of proof terms in a style reminiscent of Andromeda, except that instead of a explicitly typed language of algebraic effects and handlers, Pearl is based on a untyped programming language of realizers extended with only two opaque operations for performing reflection and explosion.

1 Introduction

Pearl is an experimental proof assistant for the formalization of mathematics in line with the computational tradition of type theory established by Constable, Bickford, Allen, Harper, Rahli, Sterling, Angiuli and many others. More precisely, the system serves as a "window on the truth" with respect to an intended realizability semantics that is defined independently of any background formal theory, viewing programs as realizers and not derivations in a fixed and necessarily incomplete proof systems. Echoing the downplay of formal systems and the insistence on human cognition advocated by Brouwer and Bishop, "computational type theory" is an umbrella term for any type theory based on this semantic view [3, 9, 14]. Pearl implements the dialect of computational type theory systematically developed by Bentzen [9] in order to provide by purely prematheматical means a philosophically robust account of the metamathematical PER semantics that underlies earlier proof assistants based on computational type theory like NuPRL [13] and its descendents [1, 18, 19]. This informal semantics is in effect a refinement of the computational version of the meaning explanations known as the "program explanations" [9] to emphasize its role as a generalization of the Brouwer–Heyting–Kolmogorov proof explanation of the intuitionistic logical connectives [30]. Pearl is built on the conviction that an informal, intuitive, prematheматical semantics is the only route to the justificatory autonomy of any putative foundation for mathematics [22, 23]. The justificatory value of theorem proving in a proof assistant is predicated on its own justification. In what follows we elaborate on the motivation behind the development Pearl, present the type theory implemented by Pearl, and document some of Pearl's features.

2 Computational type theory

The key insight of computational type theory is that computation is prior to typing. Since the computation of an untyped term need not terminate, there can be no algorithm that decides when a program realizes a type. Computationally, type membership is a semantic notion that eludes formalization [3,9]. Since semantics comes first, any rule of inference in computational type theory has to be semantically justified.[1] NuPRL and its successors navigate truth by means of a proof refinement logic, where users decompose proof goals into subgoals by means of refinement rules exclusively operating on the level of judgments to extract realizers [1, §3].

3 Motivation

Originally developed by Robert Constable and the members of the PRL group based at Cornell University in the eighties [13], NuPRL has remained the most influential proof assistant based on the computational type theory for four decades. However, as reported by Christoph Kreitz [20], the NuPRL group has in effect been disbanded and the original system is no longer so easily accessible. Of course, RedPRL [1] remains an excellent alternative, but we believe that cubical features may present a distraction from the premathematical essence of the computational type theory. While the theory is justified by a cubical generalization of the PER semantics [2], such meaning explanations rely on the mathematical concept of cubical set, so it is unclear whether they qualify as a justification in the philosophical sense [6, 9]. This is in no way meant to diminish the practical value of their work. As a matter of fact, we began in 2019 the earliest iteration of the present project [8] as an implementation of a cubical alternative to extensional type theory known as XTT [29]. After a long break devoted to research on the philosophical roots of type theory, that project was resumed in the summer of 2026 upon the completion of the monograph [9]. Pearl owes its very existence to the realization that this original implementation could be decubicalized in a practical way to match more closely the spirit of computational type theory.

4 Related work

Although inspired by NuPRL, Pearl deviates from it in three important ways. First, it is intended to be consistent with classical reasoning in adherence to Bishop's band of constructivism mathematics [11], while the latest versions of NuPRL [10, 15, 27] gradually came to favor the formalization of intuitionistic mathematics, including the accommodation of continuity principles, free choice sequences, and elements of Brouwer's creating subject arguments that prove classically false theorems [7]. We see this incompatibility with classical mathematics as a design choice that is orthogonal to the computational view of type theory per se. Indeed, the type theory of Pearl rests on an open-ended ontology of untyped programs [9, §11]. The law of excluded middle cannot be refuted because we cannot rule out the possibility that an oracle deciding whether any type is inhabitable might find its way as a program.

[1] Of course, one such rule is equality reflection [9, 17, 28]. But computational type theory is not to be confused with extensional Martin-Löf type theory [24]. The former is a semantics while the latter a formal theory that happens to feature equality reflection. In the case of our preferred form of computational type theory [9], we are dealing with an informal semantics best described as a naive type theory.

Second, Pearl operates without a proof refinement logic as NuPRL does. Pearl is designed to get around the difficulties involving accessing truth in the intended interpretation and dealing with undecidability without going too far beyond a proof term language in a style reminiscent of Andromeda [5]. But the latter is based on an explicitly typed language of algebraic effects and handlers intended to accommodate generic theories with user-definable rules of inference. This use of explicit typing and freedom from models is not suitable to computational type theory. In contrast to Andromeda, Pearl is based on a untyped programming language and specifically implements computational type theory with Curry-style types being attributed to programs depending on their computational behavior depending on two opaque term-level operations for reflection and explosion. This allows Pearl to rely on incremental proof construction with no separate tactics language much like Agda [26]. To illustrate this, we note that, in contrast to the usual status of function extensionality as a primitive rule of inference in NuPRL [13, p. 179; 21, A.3.1], we can prove function extensionality in Pearl using only equality reflection and η-conversion:

```
thm funext {A : type ℓ} {B : A → type ℓ} (f g : Π (x : A), B x) :
  (Π (x : A), eq (B x) (f x) (g x)) → eq (Π (x : A), B x) f g :=
λ h, reflect (etad f) reflect (etad g) (λ x, (reflect (h x) rfl) x)
```

It is also instructive to compare Pearl's approach with another recent implementation of equality reflection found in Istari [16], a new experimental proof assistant that is based on a version of Martin-Löf extensional type theory extended with impredicative quantification and guarded recursive types. Istari is tactic-oriented just as Rocq [4], but deals with extensional equality by means of an `extensionality` tactic. Function extensionality follows directly its application on the hypotheses.[2] Moreover, notice that, although it accommodates equality reflection, Istari is not an implementation of a formal type theory without ties to the computational view of type theory.

Third, Pearl deviates from NuPRL in its attempt to be more inclusive. This attitude is perhaps best described in analogy to how Heyting was a more flexible advocate of intuitionistic reasoning compared to Brouwer [7]. To be exact, Pearl is designed to emulate the behavior of intensional equality in proof assistants like Agda [26], Lean [25], or Rocq [4] when the full semantic expressive power of computational type theory is not needed and a typechecking strategy is available. This hybrid approach is meant to make computational type theory more accessible to users already familiar with those mainstream proof assistants, while overcoming the difficult computational challenges of working with intensional equality in practice, which often lead to the postulation of axioms that break the computational content of theorems [31]. Because Pearl relies on untyped computation to determine type membership, there is no way to verify that types are inhabited without computing the inhabitant. All of this is achieved by embracing the undecidability of type checking to its fullest. Of course, it could be argued that this is too high a price to pay. In our view, securing decidable type checking makes little difference in practice. After all, we want proof assistants to be able to elaborate omitted bits of the syntax and type inference is undecidable. Lean is an excellent example of how a proof assistant can succeed in the formalization of mathematics even without decidable judgmental equality.

[2] See https://github.com/runmingl/istari-stc/blob/main/src/Examples.ist.

5 Core language

Pearl is built on a command language for entering proof environments, among others of its many functions, a simple metalanguage for manipulating reflection and explosion on mathematical objects, and finally the object language, an open-ended untyped programming language that defines Pearl's ontology in the style of [9, §11]. This object language comes with a syntax and an operational semantics.

5.1 The object language

The object language is the point of departure of the system. It is based on the untyped lambda calculus extended with constants for a small selection of type formers and their constructors and eliminators (when they do in fact exist). The raw user language has named variables but internally Pearl uses locally nameless representation [12]. Regarding the selection, to keep the kernel small in line with the de Bruijn Criterion, Pearl only includes, in addition to user-defined inductive type families, a built-in dependent function type, equality type, quotient type, empty type, and a cumulative hierarchy of universes à la Russell. Below you can find a simplified version of the raw user syntax where the three dots indicate the possibility of further extension:

$$
\begin{aligned}
\mathsf{var} &:= x \mid y \mid z \mid \dots \\
\mathsf{expr} &:= \mathsf{var} \mid \Pi_{(\mathsf{var}:\mathsf{expr})}\mathsf{expr} \mid \lambda\,\mathsf{var}.\mathsf{expr} \mid \mathsf{expr}\ \mathsf{expr} \\
&\quad \mid \mathsf{expr} = \mathsf{expr}\ \mathsf{expr} \mid \texttt{rfl} \\
&\quad \mid \mathsf{expr}/\mathsf{expr} \mid [\mathsf{expr}]\ \mathsf{expr} \\
&\quad \mid \texttt{void} \\
&\quad \mid \texttt{type 0} \mid \texttt{type 1} \mid \dots \mid \texttt{type}\ \mathsf{var} \\
&\quad \vdots
\end{aligned}
$$

The native operational semantics is based on β-reduction alone.

- Inductive type families are defined by prescribing how to construct realizers for the type using the '`inductive`' environment, with their equality conditions and and eliminators derived automatically by the system. The language of Pearl is open-ended in the spirit of [9, §9]. Every successful inductive definition of a type family introduces new constants for the type former, constructors, and eliminators. Note that eliminators are automatically generated under the name '`foorec`' for a type '`foo`'.
- Dependent function types embody implications, functions, universal quantification, and indexed products with the given as '$\Pi(\mathtt{x}:\mathtt{M}),\mathtt{N}$', where all free occurrences of '`x`' in '`N`' are bound. The constructor is given by lambda abstraction '`x.M`', and eliminator by application '`MN`'. As usual, all free occurrences of '`x`' in '`M`' are bound in '`x.M`'. Since bound variables are represented using de Bruijin indices internally, we never distinguish between the choice of bound variables in an expression. To be exact, the syntax of Pearl is based on locally nameless representation.
- Equality type express equality statements as '$M = K\ N$' or '`eq` $K\ M\ N$' and its constructor is given by the reflexivity constant '`rfl`'. As explained in

[9, 28], there is no need for a constant witnessing its elimination rule from the computational view of type theory: the semantics justifies a more general rule of inference on the level of judgments. This is the so-called reflection rule, which states that when the equality type is inhabited the two expressions are judgmentally equal at their type. When the user types '`reflect M N`' when proving a theorem and '*M*' realizes an equality type, the elaborator rewrites '*N*' and the current goal so that the terms are syntactically indistinguishable for unification purposes. This is achieved internally using explicit substitutions to keep track of reflected terms.

- Empty type expresses absurdity and empty collections as '`void`'. This type has no constructors and no eliminators. Again, there is no need for a constant witnessing ex falso. Computationally, explosion is well-justified on the level of judgments. Thus, when the user types '`explodeMN`' when proving a theorem and '`M`' realizes the empty type, the elaborator aborts the proof leaving '`N`' left as its realizer.

- Universe types are inhabited by types. Given the propositions-as-types, correspondence, there is no distinction between sets and propositions. Universe types are written as '`type`ℓ', where 'ℓ' is the level it lives in. There is a cumulative hierarchy of them, with type formers '`type0`', '`type1`', ... There are no constructors and eliminators, as a consequence of Pearl's preferred formulation of universes à la Russell.

6 Usage

Interaction with Pearl is best done using the pearl-vscode extension for verification support. Once you are all set, note that you can use the following commands for inductively defining type families, entering the proof environment, type inference, term evaluation, importing files for the modularizion of proofs, declaring universe levels, and extracting and printing terms from definitions and theorems:

7 Inductive environment

This environment is used to introduce inductively defined type families. Inductive type declarations must be predicative, meaning that the target type must not live in a universe level that is lower than a universe level occurring as a parameter or argument. They must also be strictly positive, in that roughly the defined type never occurs in a ill-founded way. Finally, to avoid redundency, Pearl does not allow for empty inductive type declarations given that the empty type is already taken as primitive.

The syntax for defining an indexed inductive type is:

```
inductive foo [context] : type ℓ
| constructor1 : ... → foo
| constructor2 : ... → foo
...
| constructorn : ... → foo
```

8 Proof environment

The proof environment is the main command where definitions are stated and theorems and lemmas are proven. Users can use the labels ‘def‘, ‘definition‘, ‘abbrev‘, ‘abbreviation‘ for definitions and ‘thm‘, ‘theorem‘, ‘lem‘, and ‘lemma‘ for theorems. There is a subtle difference between definitions and theorems. Defined terms will always unfold at elaboration time, but not proofs of theorems.

```
def [id] [context] : [type] := [term]
```

Contexts are lists of variable declarations. Explicit variables are enclosed with ‘(‘ and ‘)‘ and must be passed as parameters when the definition or theorem is reused later. Implicit variables are enclosed with curly braces ‘‘ and ‘‘ and can be inferred by the elaborator. Multiple variables of the same type can be declared at the same time.

In addition, the user can also have ‘_‘ and ‘¿ as placeholders in any expression. The former is used to synthesize an implicit assumption or fail after a number of steps, evaluating and displaying the type of the current goal. The latter is used to display the type of current goal directly without evaluation.

9 Type inference

The syntax for type inference is:

```
infer [context] : foo
```

10 Evaluation

Pearl does not rely on normalization-by-evaluation. Instead, evaluation is untyped due to philosophical reasons having to do with closer adherence to the computational view of type theory. The syntax for term evaluation is:

```
eval foo
```

Like NuPRL, Pearl’s notion of evaluation only relies on beta-reduction. Eta-conversion is handled at elaboration time.

11 File import

This command imports a specified file into the current file. The syntax is

```
import [filename]
```

12 Universe declaration

This declares a universe level variable according to the syntax:

```
universe [id]
```

Note that if you are using the VS Code Extension you may type the small script ell ‘‘ with ‘
ell‘.

13 Term extraction

This extracts and prints the realizer corresponding to a definition or theorem with the syntax

```
print [name]
```

References

[1] Carlo Angiuli, Evan Cavallo, Kuen-Bang Hou (Favonia), Robert Harper, and Jonathan Sterling. The RedPRL proof assistant (invited paper). *Electronic Proceedings in Theoretical Computer Science*, 274:1–10, 2018. URL: http://dx.doi.org/10.4204/EPTCS.274.1.

[2] Carlo Angiuli and Robert Harper. Meaning explanations at higher dimension. *Indagationes Mathematicae*, 29(1):135–149, 2018. URL: https://doi.org/10.1016/j.indag.2017.07.010.

[3] Carlo Angiuli, Robert Harper, and Todd Wilson. Computational higher-dimensional type theory. *Proceedings of the 44th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '17*, 52(1):680–693, 2017. URL: https://www.cs.cmu.edu/~cangiuli/papers/chtt.pdf.

[4] Bruno Barras, Samuel Boutin, Cristina Cornes, Judicaël Courant, Jean-Christophe Filliatre, Eduardo Gimenez, Hugo Herbelin, Gerard Huet, Cesar Munoz, Chetan Murthy, et al. The Coq proof assistant reference manual: Version 6.1, 1997. URL: https://inria.hal.science/inria-00069968.

[5] Andrej Bauer, Gaëtan Gilbert, Philipp G. Haselwarter, Matija Pretnar, and Christopher A. Stone. Design and Implementation of the Andromeda Proof Assistant. In Silvia Ghilezan, Herman Geuvers, and Jelena Ivetic, editors, *22nd International Conference on Types for Proofs and Programs (TYPES 2016)*, volume 97 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 5:1–5:31, Dagstuhl, Germany, 2018. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. URL: https://doi.org/10.4230/LIPIcs.TYPES.2016.5.

[6] Bruno Bentzen. What types should not be. *Philosophia Mathematica*, 28(1):60–76, 2020. URL: https://doi.org/10.1093/philmat/nkz014.

[7] Bruno Bentzen. Intuitionism in mathematics. In James Fieser and Bradley Dowden, editors, *The Internet Encyclopedia of Philosophy*. 2025. URL: https://iep.utm.edu/intuitionism-math.

[8] Bruno Bentzen. Cube, 2026. Experimental proof assistant for cubical type theory with proof irrelevance. Originally named "Cubo". URL: https://github.com/bbentzen/cube.

[9] Bruno Bentzen. *Philosophical type theory*, volume 68 of *Logic, Epistemology, and the Unity of Science*. Springer, Cham, 202x. URL: https://link.springer.com/book/9783032419392.

[10] Mark Bickford, Liron Cohen, Robert L Constable, and Vincent Rahli. Computability beyond church-turing via choice sequences. In *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science*, pages 245–254, New York, NY, USA, 2018. ACM. URL: 10.1145/3209108.3209200.

[11] Errett Bishop. *Foundations of Constructive Analysis*. Academic Press, New York, 1967.

[12] Arthur Charguéraud. The locally nameless representation. *Journal of automated reasoning*, 49(3):363–408, 2012. URL: https://doi.org/10.1007/s10817-011-9225-2.

[13] R. L. Constable, S. F. Allen, H. M. Bromley, W. R. Cleaveland, J. F. Cremer, R. W. Harper, D. J. Howe, T. B. Knoblock, N. P. Mendler, P. Panangaden, J. T. Sasaki, and S. F Smith. *Implementing Mathematics with The Nuprl Proof Development System*. Prentice-Hall, 1985. URL: https://www.nuprl.org/book/.

[14] Robert L. Constable. Computational type theory. *Scholarpedia*, 4(2):7618, 2009. URL: http://dx.doi.org/10.4249/scholarpedia.7618.

[15] Robert L Constable. Computational complexity and brouwers continuum, 2020. Unpublished note. URL: https://nuprl-web.cs.cornell.edu/documents/Constable/The-Continuum.pdf.

[16] Karl Crary. The Istari proof assistant, 2025. URL: https://istarilogic.org.

[17] Peter Dybjer. Program testing and the meaning explanations of intuitionistic type theory. In Peter Dybjer, Sten Lindström, Erik Palmgren, and Göran Sundholm, editors, *Epistemology versus Ontology*, pages 215–241. Springer, 2012. URL: https://doi.org/10.1007/978-94-007-4435-6_11.

[18] Jason J. Hickey. *The MetaPRL Logical Programming Environment*. PhD thesis, Cornell University, Ithaca, NY, 2001. URL: https://dl.acm.org/doi/book/10.5555/932618.

[19] Vincent Rahli Jonathan Sterling, Daniel Gratzer. Jonprl, 2018. An proof refinement logic for computational type theory. Inspired by Nuprl. URL: https://github.com/jonsterling/JonPRL.

[20] Christoph Kreitz. Answer to “What is the state of Nuprl and Extensional Type Theory?", Proof Assistants Stack Exchange. Licensed under CC BY-SA 3.0. URL: https://proofassistants.stackexchange.com/q/2346/1048.

[21] Christoph Kreitz. The nuprl proof development system, version 5: Reference manual and users guide. *Department of Computer Science, Cornell University*, 10, 2002. URL: https://www.cs.cornell.edu/info/people/kreitz/PDF/02cucs-NuprlManual.pdf.

[22] James Ladyman and Stuart Presnell. Does homotopy type theory provide a foundation for mathematics? *The British Journal for the Philosophy of Science*, 69(2), 2016. URL: https://doi.org/10.1093/bjps/axw006.

[23] Øystein Linnebo and Richard Pettigrew. Category theory as an autonomous foundation. *Philosophia Mathematica*, 19(3):227–254, 2011. URL: https://doi.org/10.1093/philmat/nkr024.

[24] Per Martin-Löf. Constructive mathematics and computer programming. In *Logic, methodology and philosophy of science, VI (Hannover, 1979)*, volume 104 of *Stud. Logic Found. Math.*, pages 153–175. North-Holland, Amsterdam, 1982. URL: https://doi.org/10.1016/S0049-237X(09)70189-2.

[25] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *International Conference on Automated Deduction*, pages 625–635. Springer, 2021. URL: https://doi.org/10.1007/978-3-030-79876-5_37.

[26] Ulf Norell. *Dependently Typed Programming in Agda*, pages 230–266. Springer Berlin Heidelberg, Berlin, Heidelberg, 2009. doi:10.1007/978-3-642-04652-0_5.

[27] Vincent Rahli, Mark Bickford, and Robert L Constable. Bar induction: The good, the bad, and the ugly. In *2017 32nd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*, pages 1–12. IEEE, 2017. URL: https://vrahli.github.io/articles/bar-induction-lics-short.pdf.

[28] Jonathan Sterling. Type theory and its meaning explanations. Unpublished note, 2015. URL: https://arxiv.org/abs/1512.01837.

[29] Jonathan Sterling, Carlo Angiuli, and Daniel Gratzer. Cubical Syntax for Reflection-Free Extensional Equality, 2019. doi:10.4230/LIPIcs.FSCD.2019.31.

[30] Anne S. Troelstra and Dirk van Dalen. *Constructivism in mathematics. Vol. I*, volume 121 of *Studies in Logic and the Foundations of Mathematics*. North-Holland, Amsterdam, 1988.

[31] UFP (The Univalent Foundations Program). *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study, Princeton, 2013. URL: https://homotopytypetheory.org/book/.